▯ Java Collections Framework – Complete Hierarchy (Java 8+)

The **Java Collections Framework (JCF)** is part of **Java Platform, Standard Edition** and mainly lives in the java.util package.

It has:

- **Interfaces**
- **Concrete Classes (Implementations)**
- **Abstract Classes**
- **Legacy Classes**

---

1 Root Interface: Iterable

```
Iterable (interface)
  └── Collection (interface)
```

---

2 Collection Hierarchy

```
Iterable
└── Collection
    ├── List
    │   ├── ArrayList
    │   ├── LinkedList
    │   ├── Vector (Legacy)
    │   │   └── Stack (Legacy)
    │   └── CopyOnWriteArrayList (java.util.concurrent)
    │
    ├── Set
    │   ├── HashSet
    │   │   └── LinkedHashSet
    │   ├── TreeSet
    │   ├── EnumSet
    │   └── CopyOnWriteArraySet (java.util.concurrent)
    │
    └── Queue
        ├── PriorityQueue
        ├── Deque
        │   ├── ArrayDeque
        │   └── LinkedList
        │
        └── BlockingQueue (java.util.concurrent)
            ├── ArrayBlockingQueue
            ├── LinkedBlockingQueue
            ├── PriorityBlockingQueue
            ├── DelayQueue
```

```
        └── SynchronousQueue
```

---

3 Map Hierarchy (Separate from Collection)

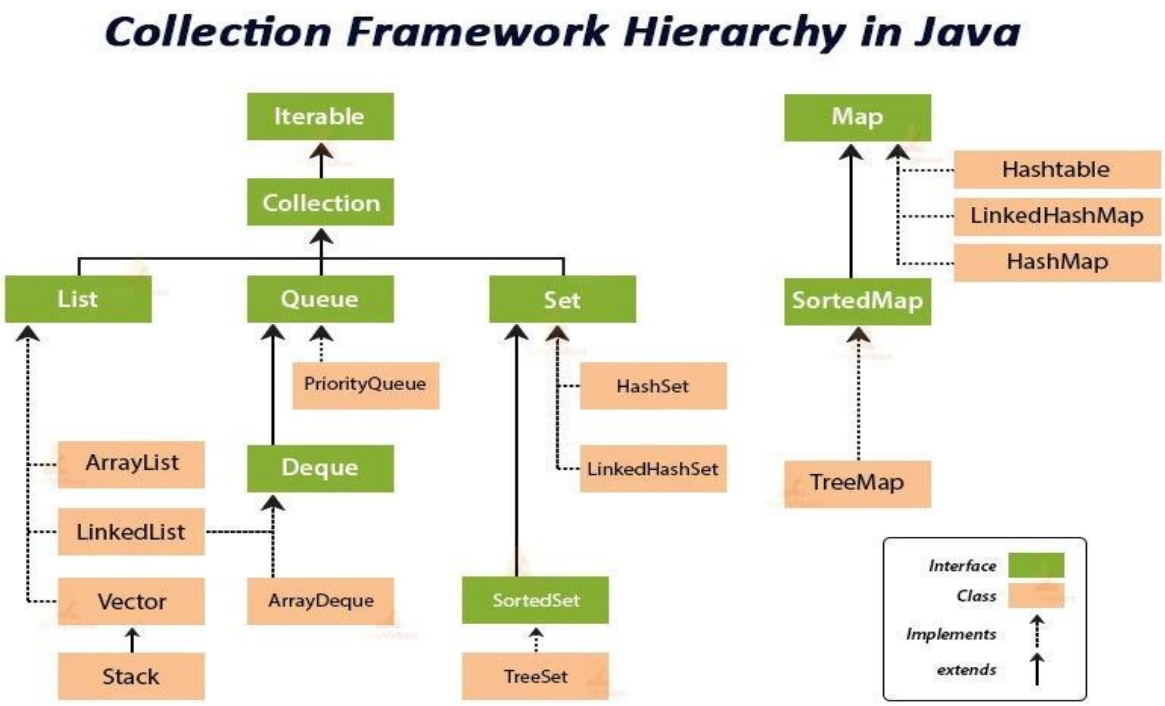
□ Map does NOT extend Collection

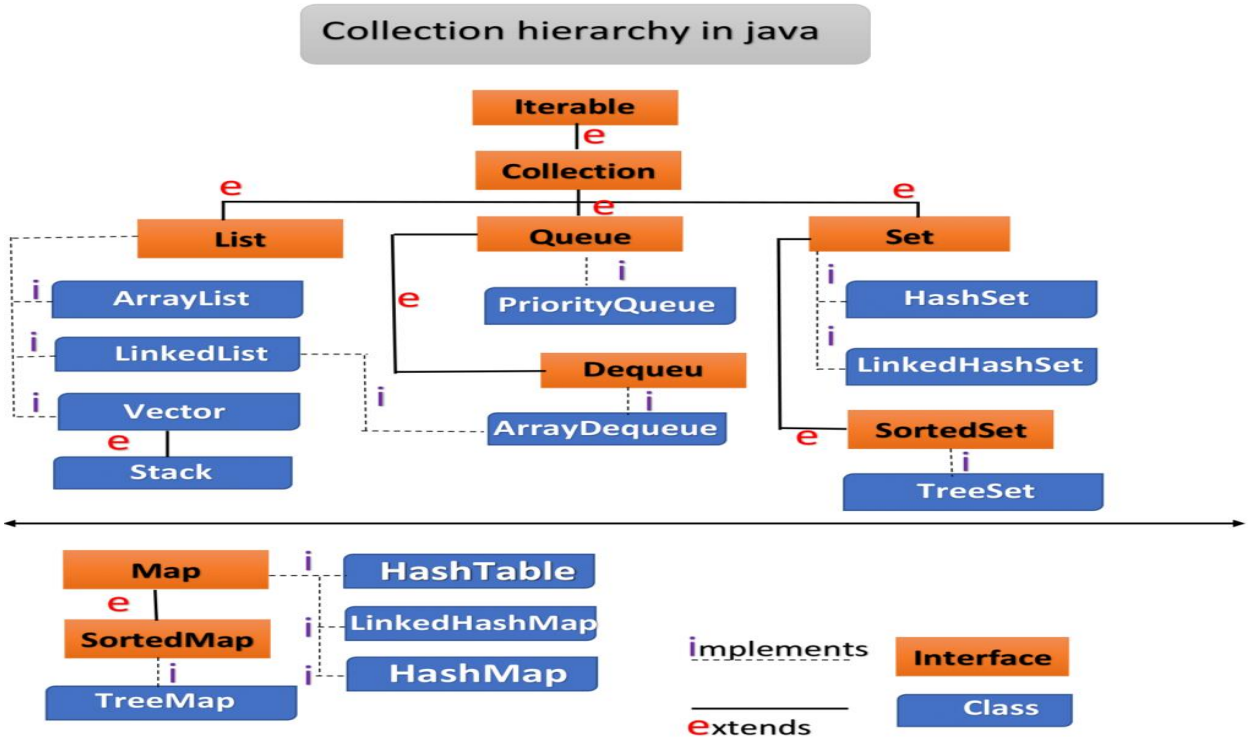
```
Map
├── HashMap
│   └── LinkedHashMap
├── TreeMap
├── Hashtable (Legacy)
│   └── Properties
├── WeakHashMap
├── IdentityHashMap
├── EnumMap
└── ConcurrentHashMap (java.util.concurrent)
```

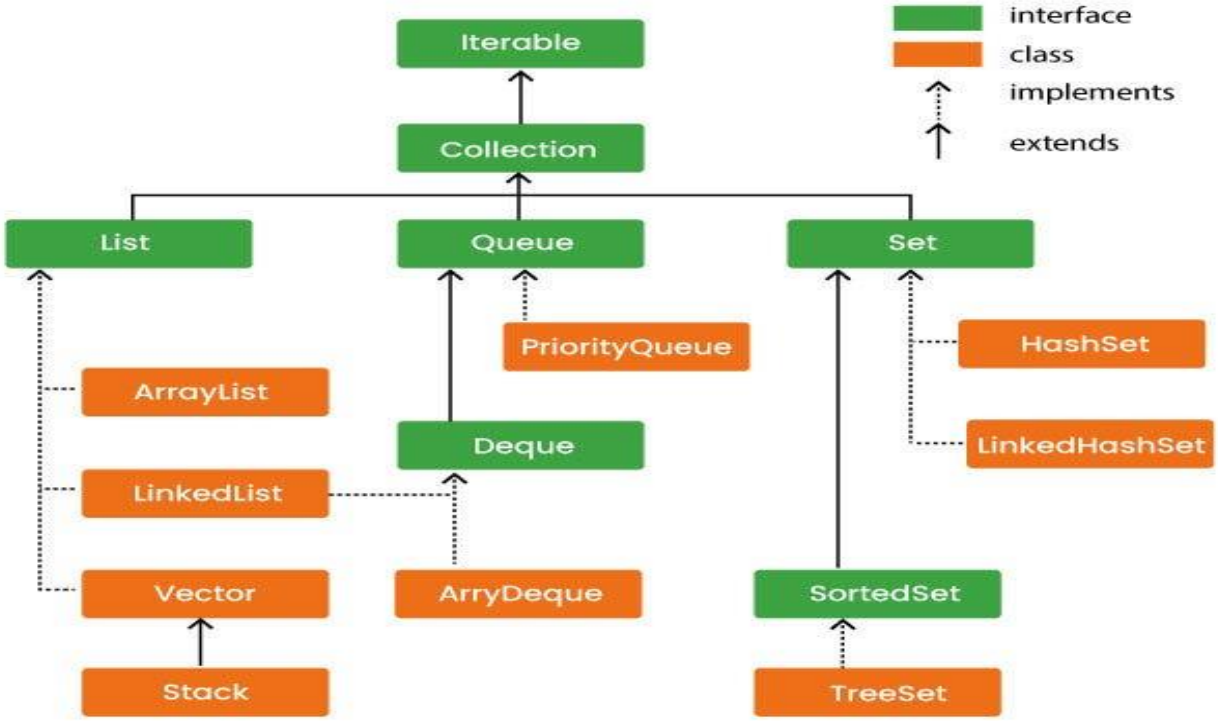
---

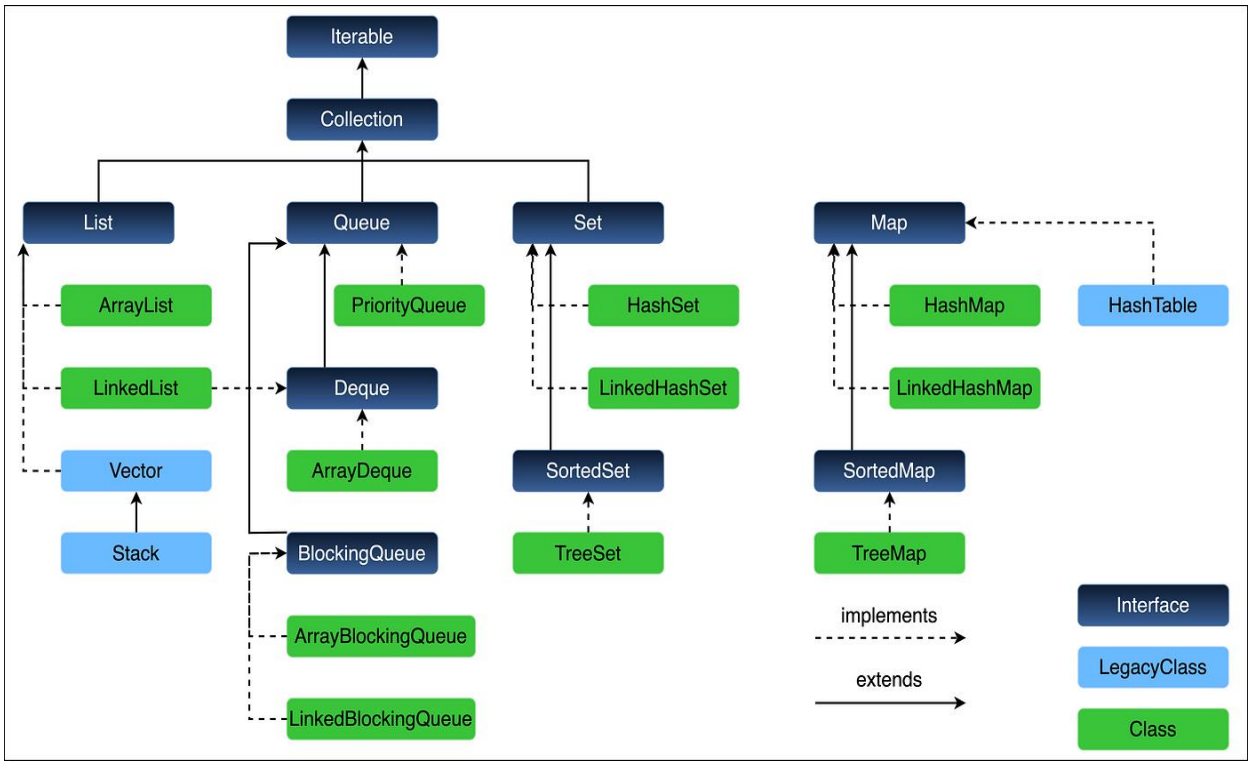
4 Visual Hierarchy Diagram

□ **Collection Hierarchy**

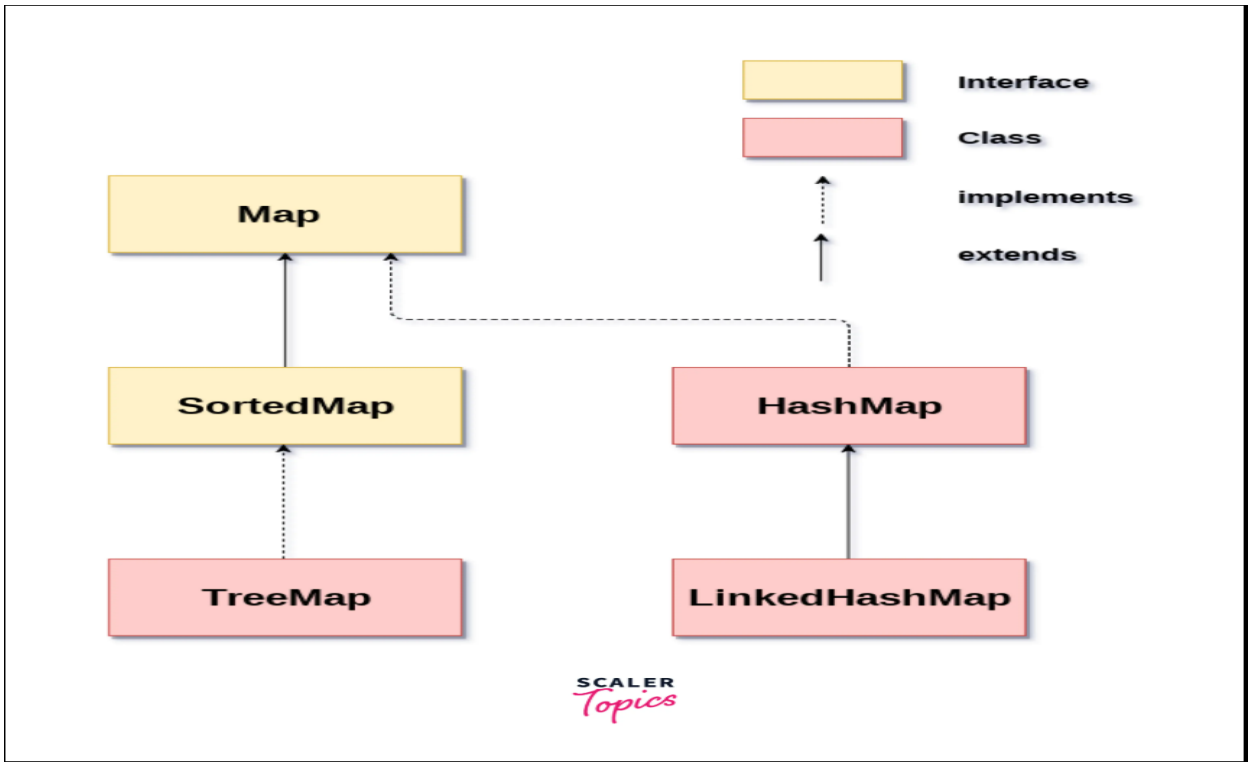


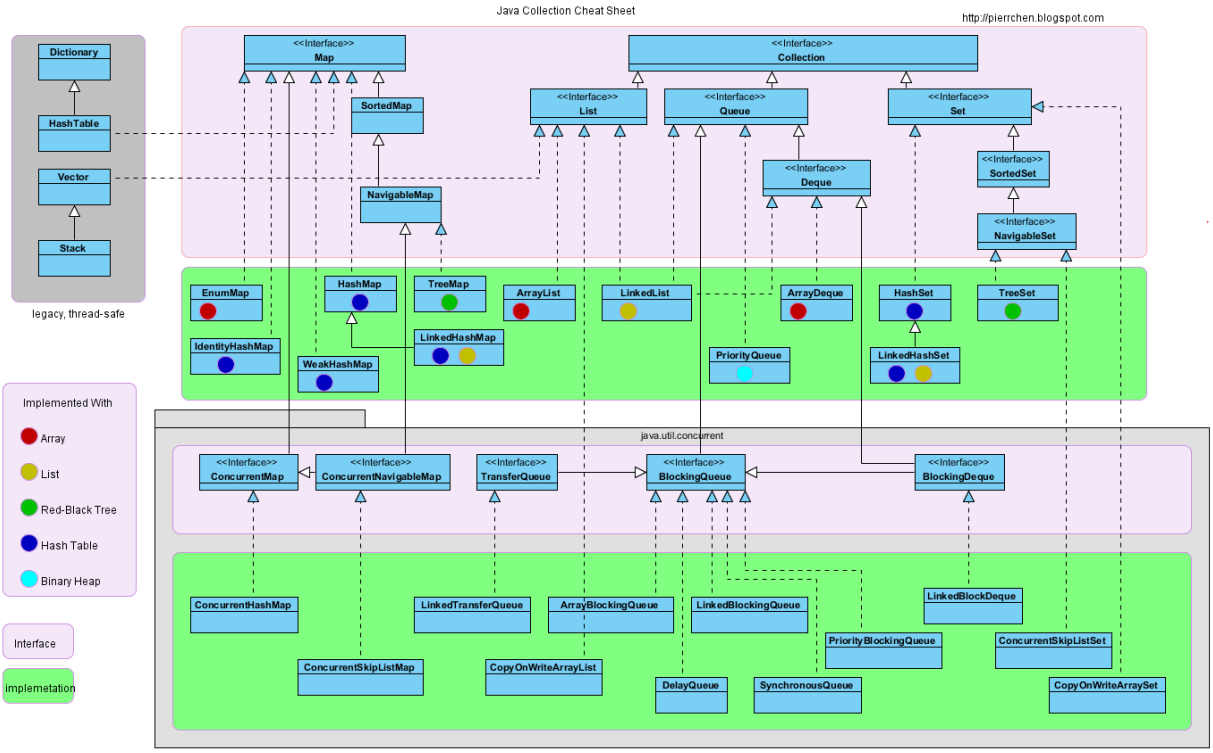

Collection hierarchy in java
Iterable
Collection
List
Queue
Set
ArrayList
LinkedList
Vector
Stack
PriorityQueue
Dequeu
ArrayDequeue
HashSet
LinkedHashSet
SortedSet
TreeSet
Map
SortedMap
TreeMap
HashTable
LinkedHashMap
HashMap
implements
extends
Interface
Class

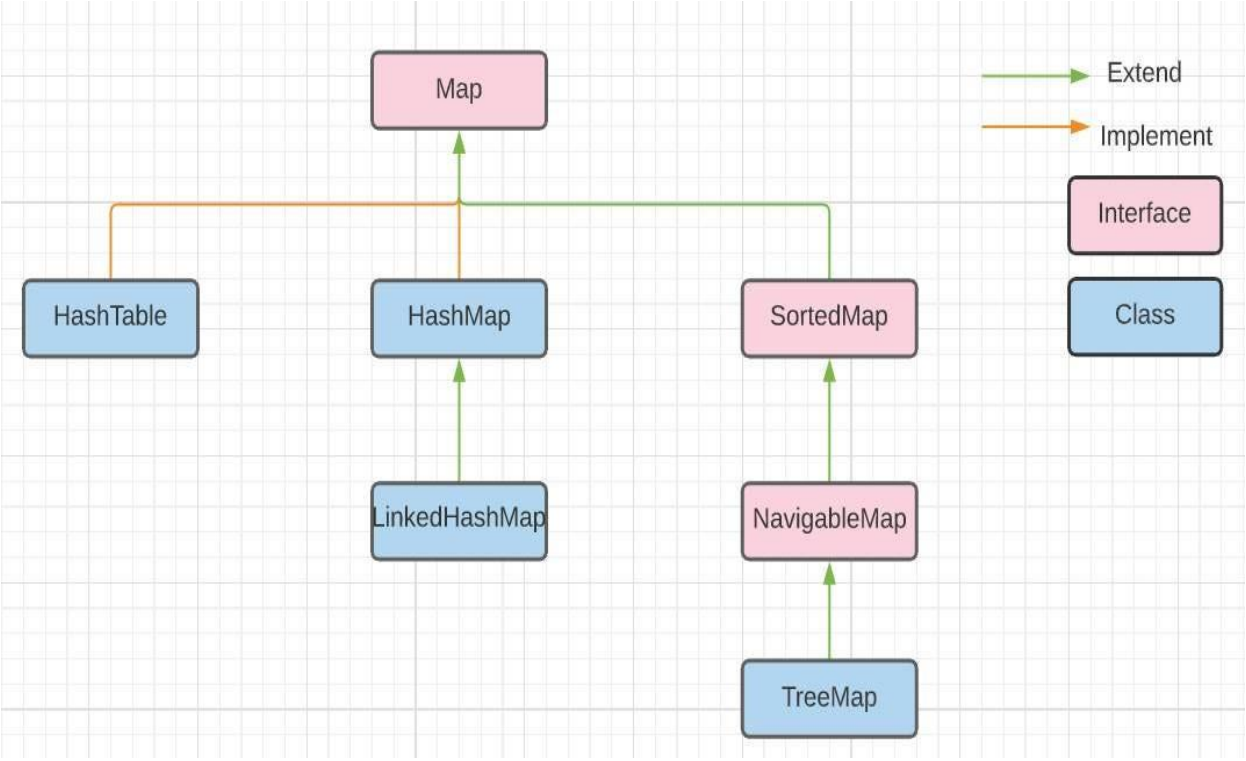

interface
class
implements
extends
Iterable
Collection
List
Queue
Set
ArrayList
LinkedList
Vector
Stack
PriorityQueue
Deque
ArryDeque
HashSet
LinkedHashSet
SortedSet
TreeSet

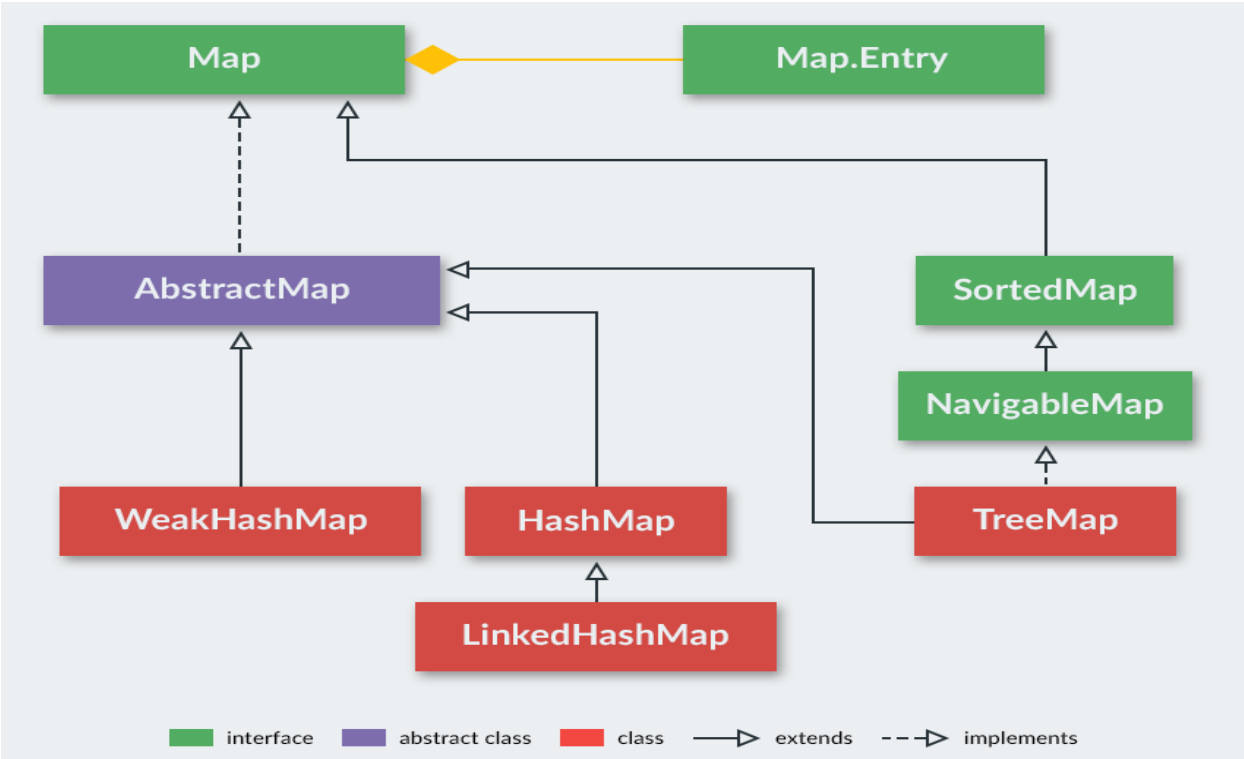


□ Map Hierarchy




Java Collection Cheat Sheet
http://pierrchen.blogspot.com
Dictionary
HashTable
Vector
Stack
legacy, thread-safe
<<Interface>> Map
<<Interface>> Collection
SortedMap
<<Interface>> List
<<Interface>> Queue
<<Interface>> Set
<<Interface>> Deque
<<Interface>> SortedSet
NavigableMap
<<Interface>> NavigableSet
EnumMap
HashMap
TreeMap
ArrayList
LinkedList
ArrayDeque
HashSet
TreeSet
IdentityHashMap
WeakHashMap
LinkedHashMap
PriorityQueue
LinkedHashSet
Implemented With
Array
List
Red-Black Tree
Hash Table
Binary Heap
Interface
implemetation
java.util.concurrent
<<Interface>> ConcurrentMap
<<Interface>> ConcurrentNavigableMap
<<Interface>> TransferQueue
<<Interface>> BlockingQueue
<<Interface>> BlockingDeque
ConcurrentHashMap
LinkedTransferQueue
ArrayBlockingQueue
LinkedBlockingQueue
LinkedBlockDeque
PriorityBlockingQueue
ConcurrentSkipListSet
ConcurrentSkipListMap
CopyOnWriteArrayList
DelayQueue
SynchronousQueue
CopyOnWriteArraySet


Map
Extend
Implement
Interface
Class
HashTable
HashMap
SortedMap
LinkedHashMap
NavigableMap
TreeMap



---

5 Complete List of Major Java Collection Classes

□ List Implementations

Class	Ordered	Duplicate	Thread Safe
ArrayList	✓	✓	✗
LinkedList	✓	✓	✗
Vector	✓	✓	✓
Stack	✓	✓	✓
CopyOnWriteArrayList	✓	✓	✓

□ Set Implementations

Class	Ordered	Sorted	Duplicate	Thread Safe

HashSet	✗	✗	✗	✗
LinkedHashSet	✓	✗	✗	✗
TreeSet	✓	✓	✗	✗
EnumSet	✗	✗	✗	✗
CopyOnWriteArraySet	✗	✗	✗	✓

□ Queue / Deque

Class	FIFO	Priority	Thread Safe
PriorityQueue	✗	✓	✗
ArrayDeque	✓	✗	✗
LinkedList	✓	✗	✗
ArrayBlockingQueue	✓	✗	✓
LinkedBlockingQueue	✓	✗	✓
PriorityBlockingQueue	✗	✓	✓

□ Map Implementations

Class	Ordered	Sorted	Thread Safe
HashMap	✗	✗	✗
LinkedHashMap	✓	✗	✗
TreeMap	✓	✓	✗
Hashtable	✗	✗	✓
ConcurrentHashMap	✗	✗	✓
WeakHashMap	✗	✗	✗
IdentityHashMap	✗	✗	✗
EnumMap	✗	✗	✗

Properties	✕	✕	✓

---

6 Important Abstract Classes

Abstract Class	Used For
AbstractCollection	Base for Collection
AbstractList	Base for List
AbstractSet	Base for Set
AbstractQueue	Base for Queue
AbstractMap	Base for Map

---

7 Legacy Classes (Before JCF)

Class	Status
Vector	Legacy
Stack	Legacy
Hashtable	Legacy
Enumeration	Legacy Iterator

---

8 Summary Tree (Text Version)

```
Iterable
└── Collection
    ├── List → ArrayList, LinkedList, Vector → Stack
    ├── Set → HashSet → LinkedHashSet
    │       → TreeSet
    │       → EnumSet
    └── Queue → PriorityQueue
            → Deque → ArrayDeque, LinkedList
```

```
Map (Separate)
├── HashMap → LinkedHashMap
├── TreeMap
├── Hashtable → Properties
├── WeakHashMap
├── IdentityHashMap
├── EnumMap
└── ConcurrentHashMap
```

If you want, I can also provide:

- ✅ Real-world usage examples
- ✅ Performance comparison table (Time Complexity)
- ✅ Internal working (HashMap, ConcurrentHashMap, TreeMap)
- ✅ Interview-ready explanation
- ✅ Java 8 Stream relation with collections

Just tell me your next requirement

Java 8 Stream Relation with Collections

Java 8 introduced **Streams** in **Java Platform, Standard Edition 8** to process data from **Collections** in a functional and declarative way.

---

1 What is the Relationship?

 Before Java 8

Collections were processed using:

- for-loop
- enhanced for-loop
- Iterator

 After Java 8

Collections can be processed using:

- stream()
- Lambda expressions
- Functional interfaces
- Method references

 Stream does NOT store data.
 It processes data coming from a Collection.

---

2 How Collections Provide Streams

All Collection interfaces (List, Set, Queue) extend Collection, which provides:

```
Stream<E> stream();
Stream<E> parallelStream();
```

Example:

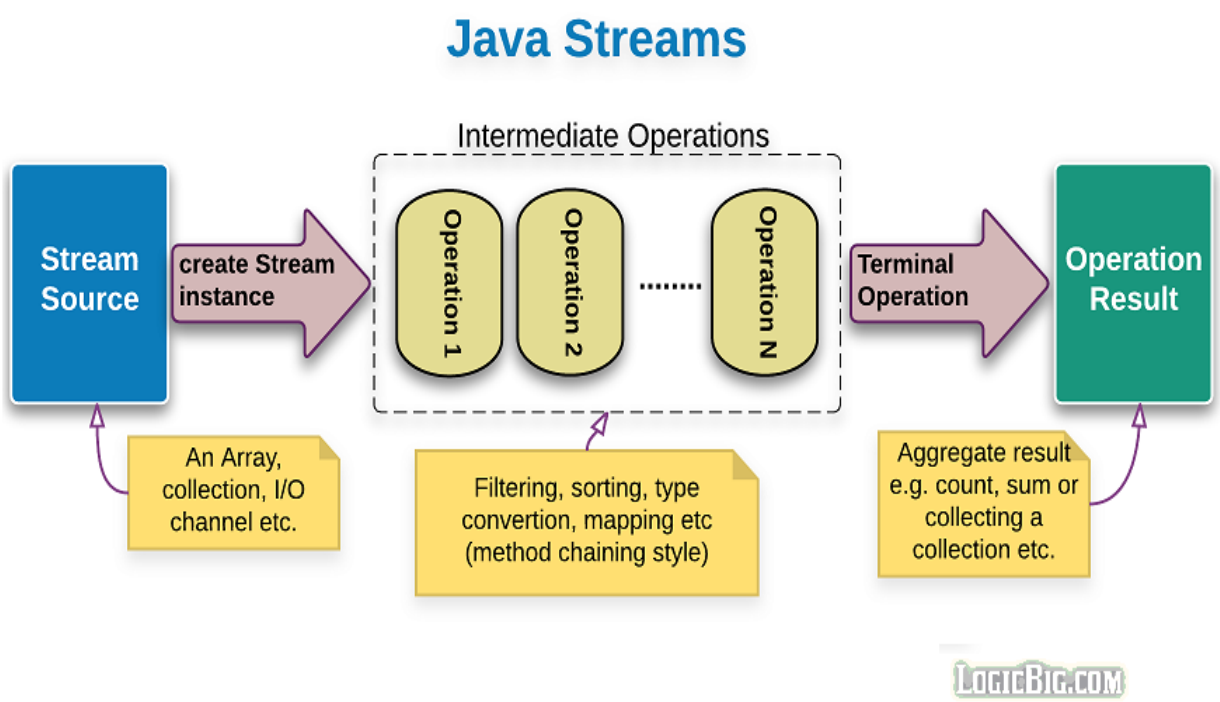
```
List<String> list = Arrays.asList("A", "B", "C");

Stream<String> stream = list.stream();
```

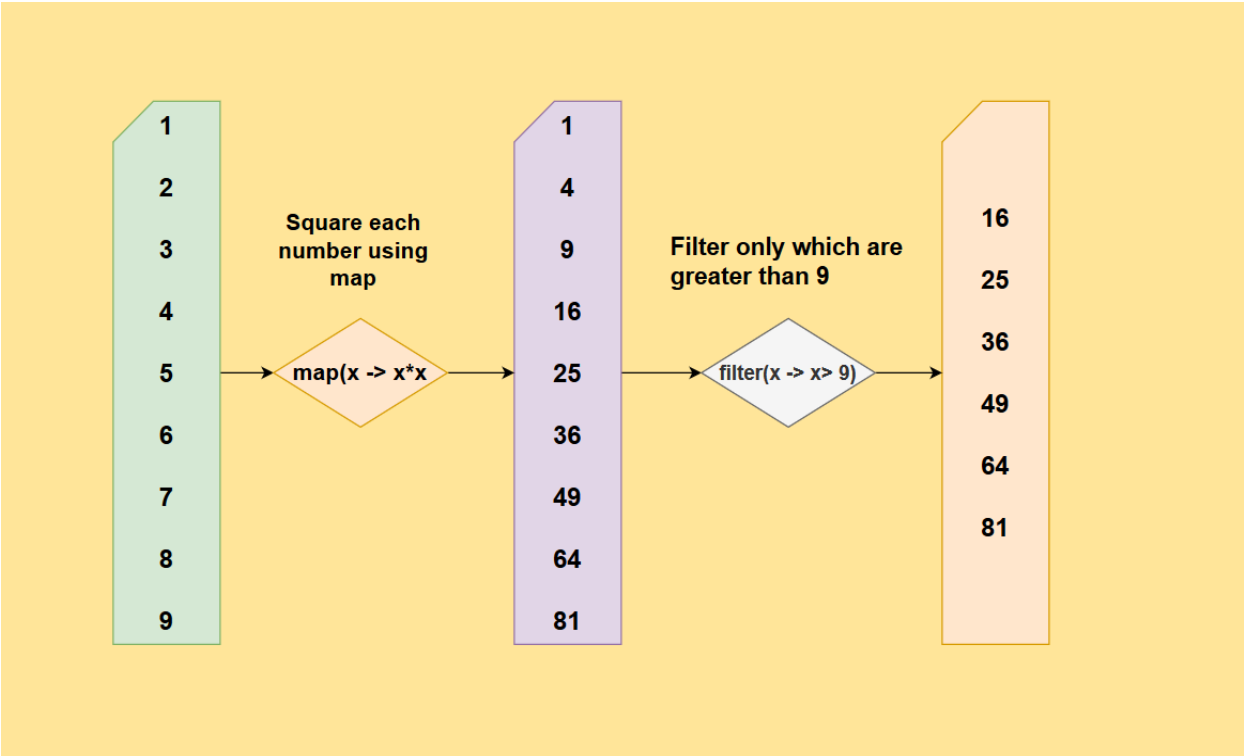
So:

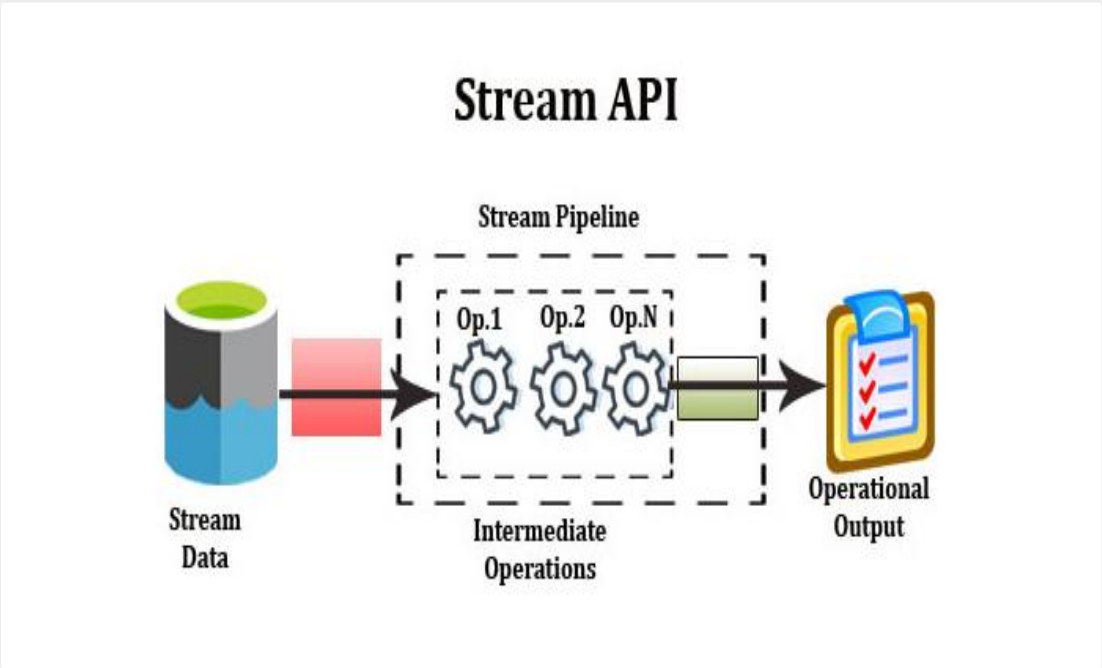
Collection → stream() → Stream API → Result

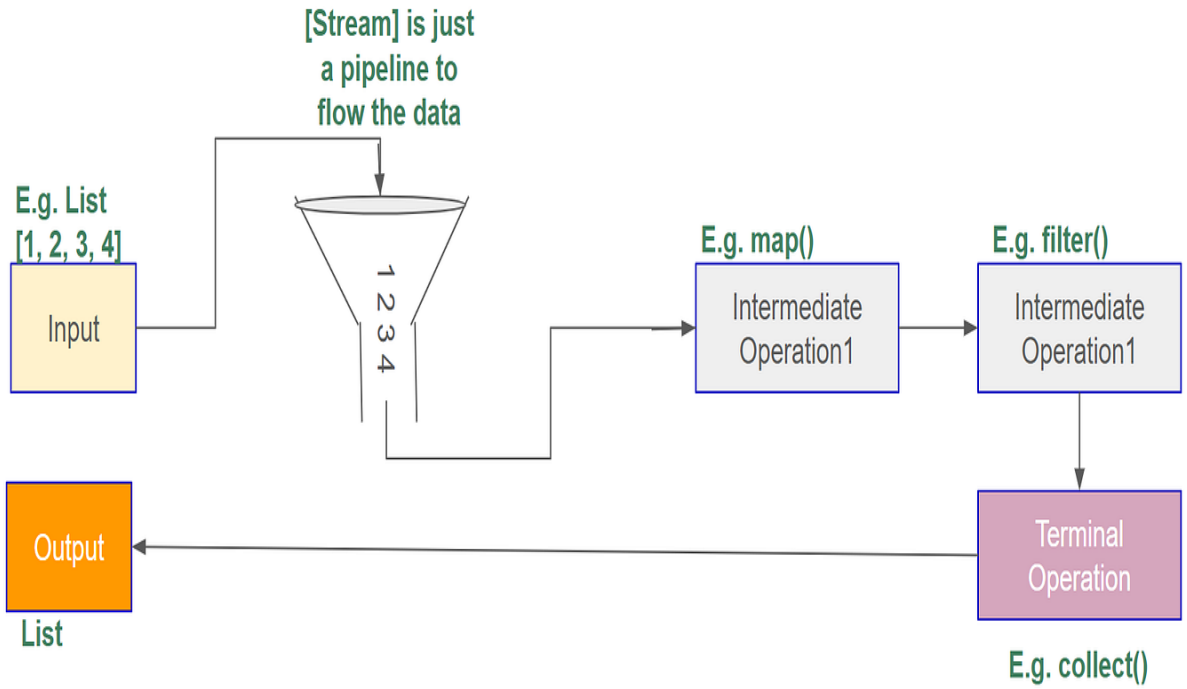
---

3 Visual Flow: Collection → Stream → Result




1
2
3
4
5
6
7
8
9
Square each number using map
map(x -> x*x
1
4
9
16
25
36
49
64
81
Filter only which are greater than 9
filter(x -> x> 9)
16
25
36
49
64
81


Stream API
Stream Pipeline
Op.1
Op.2
Op.N
Stream Data
Intermediate Operations
Operational Output



---

4 Basic Example (Collection vs Stream)

✅ Without Stream (Traditional)

```
List<Integer> numbers = Arrays.asList(10, 20, 30, 40);

List<Integer> evenList = new ArrayList<>();

for (Integer n : numbers) {
   if (n % 2 == 0) {
      evenList.add(n);
   }
}
```

✅ With Stream (Java 8)

```
List<Integer> numbers = Arrays.asList(10, 20, 30, 40);

List<Integer> evenList = numbers.stream()
      .filter(n -> n % 2 == 0)
      .collect(Collectors.toList());
```

✔ Less code

✔ More readable

✔ Functional style

---

5 Stream Processing Phases

Stream pipeline has 3 stages:

Stage	Description	Example
1 Source	Collection	list.stream()
2 Intermediate	Transform	filter(), map()
3 Terminal	Produce result	collect(), forEach()

---

6 Most Used Stream Operations on Collections

 Filtering

```
list.stream()
   .filter(x -> x > 10)
   .collect(Collectors.toList());
```

 Mapping (Transformation)

```
list.stream()
   .map(x -> x * 2)
   .collect(Collectors.toList());
```

 Sorting

```
list.stream()
   .sorted()
   .collect(Collectors.toList());
```

 Counting

```
long count = list.stream()
```

```
        .filter(x -> x > 10)
        .count();
```

 Finding Max

```
Optional<Integer> max = list.stream()
                .max(Integer::compareTo);
```

---

7 Stream with Different Collections

 List

```
List<String> names = Arrays.asList("Ram", "Shyam", "Mohan");

names.stream()
    .filter(name -> name.startsWith("R"))
    .forEach(System.out::println);
```

 Set

```
Set<Integer> set = new HashSet<>(Arrays.asList(5, 2, 8, 1));

set.stream()
  .sorted()
  .forEach(System.out::println);
```

 Map (Special Case)

Map does NOT extend Collection.

We use:

- keySet().stream()
- values().stream()
- entrySet().stream()

```
Map<String, Integer> map = new HashMap<>();
map.put("A", 10);
map.put("B", 20);

map.entrySet().stream()
  .filter(entry -> entry.getValue() > 10)
  .forEach(System.out::println);
```

---

8 Internal Iteration vs External Iteration

Feature	For Loop	Stream
Iteration	External	Internal
Parallel Support	Manual	Built-in
Readability	Medium	High
Functional	❌	✅

---

9 Parallel Streams

Collections can be processed in parallel:

```
list.parallelStream()
   .filter(x -> x > 10)
   .forEach(System.out::println);
```

✔ Uses ForkJoinPool

✔ Good for large data

⚠ Not always faster

□ Stream vs Collection (Key Difference)

Collection	Stream
Stores data	Processes data
Mutable	Immutable pipeline
Can iterate multiple times	Single-use
In-memory	Can process infinite data

---

11 Real Industry Example

Suppose you have 1 lakh employee records in List<Employee>

Find:

- Employees with salary > 50,000
- Sort by name
- Collect to new list

```
List<Employee> result = employees.stream()
    .filter(emp -> emp.getSalary() > 50000)
    .sorted(Comparator.comparing(Employee::getName))
    .collect(Collectors.toList());
```

This replaces 20+ lines of traditional code.

---

12 Important Interview Points

- Stream is introduced in Java 8
- It supports functional programming
- Lazy evaluation (intermediate ops)
- Terminal operation triggers execution
- Cannot reuse stream
- Does not modify original collection

 Summary

Collection → stream() → filter/map/sort → collect()

✔ Streams make collection processing:

- Cleaner
- Faster (parallel)
- More expressive
- Functional

If you want next:

- Deep internal working of Stream
- How Streams use Predicate, Function, Consumer
- Performance comparison
- Real-world enterprise project example
- Stream + Optional deep explanation

Tell me what you want next